Tribhuvan University
Institute of Science & Technology
Bachelor of Science in Computer Science & IT
5th Semester – Design and Analysis of Algorithms Lab

Name: Binesh	**Roll No:** 023bscit009	**Lab No:** 8

LAB 8: IMPLEMENT KRUSKAL'S ALGORITHM FOR MINIMUM SPANNING TREE (MST)

1. THEORY

Kruskal's algorithm is a classic **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph. An MST is a subset of the graph's edges that connects all vertices together without any cycles and with the minimum possible total edge weight.

The algorithm operates on the fundamental greedy principle: at every step, select the globally cheapest available edge that does not form a cycle with the edges already chosen. It continues this process until exactly **(V – 1)** edges have been selected, forming the spanning tree for **V** vertices.

To detect cycles efficiently, Kruskal's algorithm relies on the **Union-Find (Disjoint Set Union)** data structure. Each vertex is initially its own set (component). When an edge *(u, v)* is considered, the algorithm checks whether *u* and *v* belong to the same set using the **FIND** operation. If they do not, the edge is safe to include (no cycle), and the two sets are merged using the **UNION** operation. With **path compression** and **union by rank**, the Union-Find operations run in nearly O(1) amortised time.

Time & Space Complexity:

Case	Time Complexity
Best Case	O(E log E) or O(E log V)
Average Case	O(E log E) or O(E log V)
Worst Case	O(E log E) or O(E log V)
Space Complexity	**O(V + E)**

The dominant factor in the time complexity is **sorting all edges**, which takes O(E log E). Since E can be at most V^2, log E ≤ 2 log V, making O(E log E) equivalent to O(E log V). This makes Kruskal's algorithm especially efficient for **sparse graphs** where E ≈ V.

2. ALGORITHM

1. Sort all edges in **non-decreasing order** of their weight.
2. Initialise an empty set **MST** = {} and create a separate disjoint set for each vertex.
3. Pick the **smallest edge** from the sorted list.
4. Check using FIND if the two endpoints of this edge belong to **different sets** (i.e., including this edge will not form a cycle).
5. If no cycle is formed, **add the edge to MST** and UNION the two sets.
6. If a cycle would be formed, **discard** the edge.

7. Repeat steps 3–6 until the MST contains exactly **(V – 1) edges**.
8. Return the MST and its total minimum cost.

3. PSEUDOCODE

```
BEGIN SORT edges in non-decreasing order of weight CREATE empty MST = {} FOR each vertex v in Graph DO MAKE-SET(v) // each vertex is its own component END FOR FOR each edge (u, v) in sorted edge list DO IF FIND-SET(u) != FIND-SET(v) THEN ADD edge (u, v) to MST // safe: no cycle formed UNION(u, v) // merge the two components END IF END FOR RETURN MST END
```

4. SOURCE CODE

```
#include #include struct E { int u, v, w; }; // Edge structure: source, dest, weight /* Find with path compression */ int find(int p[], int i) { return p[i] == i ? i : (p[i] = find(p, p[i])); } /* Comparator for qsort – sorts edges by weight ascending */ int cmp(const void* a, const void* b) { return ((struct E*)a)->w - ((struct E*)b)->w; } void kruskal(struct E edges[], int V, int E_count) { int p[V], e = 0, cost = 0; for (int i = 0; i < V; i++) p[i] = i; // Initialise parent array qsort(edges, E_count, sizeof(struct E), cmp); printf("Path Traversed:\n"); for (int i = 0; i < E_count && e < V - 1; i++) { int x = find(p, edges[i].u); int y = find(p, edges[i].v); if (x != y) { // Different components – no cycle printf("%d -> %d (Cost: %d)\n", edges[i].u, edges[i].v, edges[i].w); p[x] = y; // Union cost += edges[i].w; e++; } } printf("Total Minimum Cost: %d\n", cost); } int main() { printf("\n023bscit009_binesh\n\n"); /* Graph edges: {source, destination, weight} */ struct E edges[] = { {0, 1, 10}, {0, 2, 6}, {0, 3, 5}, {1, 3, 15}, {2, 3, 4} }; kruskal(edges, 4, 5); return 0; }
```

5. CONCLUSION

Kruskal's algorithm successfully identified the **Minimum Spanning Tree** of the given weighted, undirected graph through a systematic greedy approach. The algorithm correctly sorted all five edges by weight and selected three edges — **2→3 (cost 4)**, **0→3 (cost 5)**, and **0→1 (cost 10)** — yielding a total minimum cost of **19**, which connects all four vertices without forming any cycle.

The use of the **Union-Find data structure with path compression** ensures that cycle detection is performed in near-constant amortised time, making the overall algorithm highly efficient. The uniform time complexity of **O(E log E)** across all cases makes Kruskal's approach particularly well-suited for **sparse graphs**, where the number of edges E is close to the number of vertices V.

This experiment demonstrates the practical power of combining a **greedy strategy** with an efficient supporting data structure. Kruskal's algorithm has widespread real-world applications including minimum-cost network wiring, road connectivity planning, cluster analysis in machine learning, and designing telecommunication networks.